HLD Level 2.9

Design a URL Shortener — Complete Interview Question & Answer

> **Interview question: Design a URL shortening service. Users submit long URLs and receive short links. Visiting a short link must redirect quickly and reliably to the original URL. The system should scale to billions of stored URLs and very high redirect traffic.**

1. Functional Requirements

Create a short URL from a long URL; redirect short code to original URL; optionally support custom aliases, expiration, deletion/disable, and basic click analytics.

2. Non-Functional Requirements

Very low redirect latency, high availability, horizontal scalability, durable mappings, globally unique short codes, abuse protection and much higher read capacity than write capacity.

3. API

```
POST /urls
{ long_url, optional_custom_alias, optional_expiry }
-> { short_url }

GET /{short_code}
-> HTTP redirect to long_url
```

4. Capacity Assumptions

```
New short URLs: 100 million/month
~= 3.33 million/day
~= 39 writes/sec average

Assume redirect:write ratio = 100:1
Redirects ~= 3,900/sec average
Peak may be 5x-10x higher
```

The exact assumptions can change. The important interview observation is that this is normally a read-heavy workload.

5. Storage Estimate

```
100 million URLs/month
5 years = 6 billion URL mappings

Assume average record ~500 bytes
6B x 500 bytes ~= 3 TB raw
plus indexes + replicas + metadata
```

The dataset is large but manageable through partitioning and replication. Redirect throughput and hot-key behavior are often more important than raw capacity.

6. High-Level Architecture

```
CLIENTS
|
v
Load Balancer
|
+----------+----------+
v v
Shorten Service Redirect Service
| |
v v
ID Generator Cache
| |
```

```
+----------+----------+
v
URL Store
|
v
Replicas/Shards
```

7. Data Model

```
short_code -> {
long_url,
created_at,
expiry,
owner_id,
status
}
```

The primary redirect lookup is short_code -> long_url, so storage should make this lookup efficient.

8. Short-Code Character Set

A common alphabet is Base62: 0-9, a-z, A-Z. Each additional character multiplies the namespace by 62.

```
6 chars -> 62^6 ~= 56.8 billion combinations
7 chars -> 62^7 ~= 3.52 trillion
```

A 7-character Base62 namespace is ample for billions of links, although reserved/blocked codes and operational choices reduce usable space.

9. Approach A: Random Code

```
generate random Base62 code
-> check whether code exists
-> if collision, retry
```

Simple and hard to predict, but requires collision handling. As occupancy rises, retries become more likely.

10. Approach B: Unique Numeric ID + Base62

```
unique integer ID = 12583920
|
v
Base62 encode
|
v
short code
```

This guarantees uniqueness if the numeric ID generator is unique. The challenge shifts to generating IDs at scale without a central bottleneck.

11. Distributed ID Generation

Options include allocating ID ranges to servers, using a timestamp + worker + sequence scheme, or using a database/coordination service to allocate ranges.

```
Server A gets IDs 1,000,000-1,999,999
Server B gets IDs 2,000,000-2,999,999
```

Range allocation avoids a network call to a central ID service for every URL creation.

12. Predictability

Sequential IDs encoded in Base62 can expose creation volume and make nearby codes guessable. If this matters, obfuscate/permutate IDs or use sufficiently random codes. Do not claim sequential Base62 is inherently secure.

13. Collision Handling

With random generation, enforce a uniqueness constraint in storage. The database is the final authority; a pre-check alone is race-prone because two servers can observe the same code as free.

14. Custom Aliases

```
example: short.domain/interview
```

Custom aliases require uniqueness checks and reserved-word filtering. They may also need ownership/authentication and stricter abuse rules.

15. Create Flow

```
Client
-> Shorten API
-> validate URL
-> generate unique code
-> write mapping durably
-> optionally warm cache
-> return short URL
```

Return success only after the mapping reaches the required durability point; otherwise a client can receive a short URL that immediately fails.

16. Redirect Flow

```
Browser -> GET /abc123
|
v
Redirect Service
|
cache lookup
/ \
HIT MISS
| |
| URL Store
| |
+----- cache fill
|
v
HTTP redirect
```

17. 301 vs 302

301 Permanent: clients/CDNs may cache aggressively, reducing backend load but making destination changes harder to observe/control.
302 Temporary: redirect service sees more traffic and retains more control/analytics. Choice depends on product semantics.

18. Why Cache?

Popular short URLs may be redirected millions of times. Caching short_code -> long_url prevents every click from hitting storage.

19. Cache-Aside

```
1. check cache
2. hit -> redirect
3. miss -> read DB
4. put mapping in cache
5. redirect
```

20. Cache Eviction

Use TTL and an eviction policy such as LRU/LFU depending on workload. Since access can be extremely skewed, a relatively small cache may serve a large fraction of redirects.

21. Negative Caching

Cache 'not found' results briefly so repeated requests to invalid/abusive codes do not hammer storage. Keep negative TTL limited so newly created links do not remain falsely missing.

22. Hot URLs

A viral link can become a hot key. Replicated/distributed cache and CDN/edge caching can absorb traffic. A single cache node should not become the only owner of an extremely popular key.

23. Database Choice

The dominant operation is key-value lookup by short code, so a distributed key-value or wide-column style store can fit well. A relational database can also work at smaller scale. Explain the access pattern rather than choosing a database by fashion.

24. Partitioning

```
shard = hash(short_code) % N
```

Hash partitioning distributes codes relatively evenly. Range partitioning on sequential IDs can create a hot newest shard unless ranges are distributed deliberately.

25. Replication

Replicate mappings for availability and read scale. Redirects can often tolerate reading a replica after creation, but newly created links may require read-after-write behavior so they work immediately.

26. Read-After-Write

A user expects a just-created short URL to redirect immediately. Solutions include reading from the leader briefly, warming cache on create, or using storage consistency settings that guarantee the needed visibility.

27. Expiration

Store an expiry timestamp. Redirect service rejects expired mappings. Background cleanup can remove expired records later; correctness should not depend on cleanup running exactly on time.

28. Deletion / Disable

A mapping can be marked disabled/tombstoned before physical deletion. Invalidate relevant cache entries so disabled links do not continue redirecting from stale cache.

29. Cache Consistency

For immutable mappings, caching is simple. If destinations can change, use explicit invalidation/versioning or short enough TTLs. Immutability is often a useful product simplification.

30. Analytics

Do not synchronously write a database analytics row on the redirect hot path. Emit a lightweight click event asynchronously.

```
Redirect Service -> redirect response
|
+-> async click event -> broker -> analytics workers/store
```

If analytics fails temporarily, redirects should normally continue.

31. Analytics Fields

```
short_code, timestamp, coarse region, referrer, user-agent category
```

Collect only what the product needs and apply privacy/security policies. Analytics is secondary to redirect availability.

32. Abuse Protection

Validate allowed URL schemes, block malicious destinations according to policy, rate-limit creation, authenticate higher-volume creators and detect spam/phishing abuse. The redirect service itself should not fetch the destination URL during normal redirect handling.

33. Availability

Redirect is the critical read path. Run stateless redirect servers behind load balancers across failure domains, use replicated cache/storage and design graceful degradation for analytics or nonessential dependencies.

34. Database Failure

If storage is temporarily unavailable, cached mappings can continue serving. Cache misses may fail, so storage replication/failover remains important. Do not silently redirect to an incorrect destination.

35. Cache Failure

If cache fails, traffic shifts to storage and can cause a thundering herd. Use cache replicas, request coalescing where useful, rate protection and ensure the database has headroom for partial cache loss.

36. Cache Stampede

When a very hot key expires, thousands of requests may miss simultaneously. Use request coalescing/single-flight, TTL jitter or proactive refresh for very hot entries.

37. Multi-Region

Redirect traffic can be served near users using regional services/caches. URL creation needs a globally unique code and durable replication. Decide the consistency requirement for newly created links across regions.

38. C++ Redirect Service

```
socket/event loop
-> HTTP parse
-> short-code validation
-> cache lookup
-> async DB lookup on miss
-> construct redirect response
-> async analytics event
```

Keep the hot path short. Avoid blocking worker threads on slow dependencies where the networking framework supports asynchronous I/O. Use connection pools, bounded queues, timeouts and RAII.

39. Memory Ownership

Cache entries and request buffers should have clear lifetimes. Prefer explicit ownership and avoid unnecessary string copies in the hot path, but optimize only after profiling.

40. Observability

Track redirect QPS, create QPS, P50/P95/P99 redirect latency, cache hit ratio, DB latency/errors, 404/expired/disabled rate, hot keys, cache evictions, analytics lag, ID-generation failures and per-region availability.

41. Complete Architecture

```
CLIENTS
|
v
Load Balancer
|
+-------------+-------------+
v v
Shorten Service Redirect Service
| |
v v
ID Generator Distributed Cache
| / \
| HIT MISS
| | |
+----------------------+-------------+
v
URL Data Store
/ | \
Shard0 Shard1 ShardN
|
Replication

Redirect Service -> async click events -> Broker -> Analytics
```

42. Common Interview Follow-Ups

How generate unique codes? Random Base62 + uniqueness constraint, or globally unique numeric IDs encoded as Base62.

Why not use a hash of the long URL? Hashes can collide, fixed short prefixes still need collision handling, and identical long URLs may or may not be intended to share a short code.

Why cache? Redirects are read-heavy and often highly skewed.

What if a link goes viral? Distributed cache/CDN and avoid a single hot owner.

What if cache fails? Storage absorbs misses; protect against thundering herd and maintain DB headroom.

How handle analytics? Emit asynchronously so redirect latency/availability does not depend on analytics.

301 or 302? Explain caching/control trade-off.

How ensure a newly created URL works immediately? Read-after-write guarantee, leader read or cache warm.

43. Interview-Ready Spoken Answer

"I'd model the core mapping as short_code to long_url and separate the write path from the much hotter redirect path. For code generation, I could use a globally unique numeric ID encoded in Base62 or random Base62 codes with a storage uniqueness constraint; the choice depends on predictability and operational requirements. The shorten service validates the URL, generates the code and persists the mapping before returning it. Redirect services are stateless and first check a distributed cache, falling back to a partitioned replicated URL store on a miss. Because redirect traffic is read-heavy and can be extremely skewed, caching and hot-key handling are important. I'd hash-partition by short code, replicate storage for availability, and ensure read-after-write behavior so a newly created link works immediately. Click analytics should be emitted asynchronously to a broker rather than written synchronously on the redirect path. I'd handle cache stampedes, negative caching, expiration and cache invalidation for disabled links. In a C++ redirect service, I'd keep the hot path short, use bounded resources, connection pools/timeouts, RAII and asynchronous I/O where appropriate, and measure cache hit ratio plus P95/P99 redirect latency."

44. Five Decisions to Remember

1. **Short-code uniqueness -> BASE62 + UNIQUE ID or RANDOM CODE + UNIQUE CONSTRAINT.**
2. **Redirect traffic is read-heavy -> CACHE FIRST.**
3. **Billions of mappings -> HASH PARTITION + REPLICATION.**
4. **Analytics is not on critical redirect path -> ASYNC EVENT.**
5. **Viral links/cache expiry -> HOT-KEY HANDLING + STAMPEDE PROTECTION.**

45. Final Revision Flow

```
Requirements
-> read/write ratio + capacity
-> APIs/data model
-> short-code generation
-> create flow
-> redirect flow
-> cache
-> database partition/replication
-> hot URLs/stampede
-> expiration/deletion
-> analytics
-> failures/multi-region
-> observability
-> C++ redirect hot path
```